UNIVERSITÉ DE SHERBROOKE
DÉPARTEMENT D'INFORMATIQUE

IFT 339

Travail pratique#4 : Ensemble avec arbre lexicographique

Dans ce travail, nous implémentons un ensemble avec un **arbre lexicographique**, une structure de données alternative aux ABR. Quelques modalités :

— Ce travail compte pour **5 %** de la session.

— La date de remise de ce travail est le **vendredi 21 novembre** jusqu'à 23h59.
Vous devez remettre sur `https://turnin.dinf.usherbrooke.ca/`.

Partie 1 : implémentation de l'ensemble

Un *arbre lexicographique* permet de stocker des objets, que l'on va appeler des *clés*, qui sont représentées par des *tuples*. Un tuple est simplement une séquence d'objets du même type. Voici des exemples de tuples :
Un tuple de int : (3, 8, 12, 4, 6, 2, 8)
Un tuple de char : (s, a, l, u, t)
Un tuple de string : ("allo", "salut", "bonjour")

Notez que l'ordre est important et qu'un même objet peut être répété dans le tuple. En C++, on peut représenter un tuple avec une structure de données de liste, par exemple avec vector ou une list. Un objet string est considéré comme un tuple dont le type d'objet est char.

Un **arbre lexicographique** permet de stocker un ensemble de tuples. Le type de tuple est au choix des utilisateurs. On peut donc déclarer des variables comme

```
arbre_lexico<vector<int>> ensemble_de_tuple_de_int;
arbre_lexico<string> ensemble_de_string;
```

Une illustration d'un arbre_lexico<string> est fournie à la page suivante. Je vous invite aussi à les visualiser avec : `https://www.cs.usfca.edu/~galles/visualization/Trie.html`.

L'important est que le type utilisé **permet l'itération**. Donc, les types utilisés doivent définir begin et end, qui retournent un itérateur qui implémente les opérateurs * et ++. Les classes vector, list, deque supportent toutes ces opérations, de même que la classe string.

Dans un arbre lexicographique, chaque noeud correspond à un objet individuel pouvant être contenu dans un tuple. Un chemin de la racine à un noeud correspond à un tuple, obtenu en concaténant les objets rencontrés dans les noeuds du chemin. Notez que le chemin qui contient seulement la racine correspond à un tuple vide.

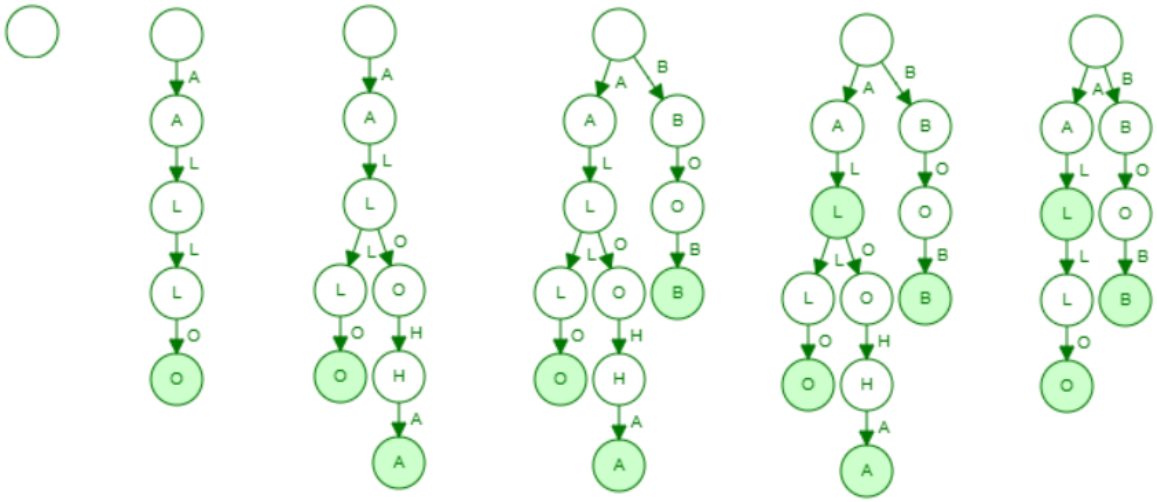

FIGURE 1 – L'état attendu d'un arbre qui stocke des string après des insertions et suppressions. Les noeuds en vert sont ceux qui on est_fin = true. À gauche, l'état initial (seule la racine existe). Ensuite, de gauche à droite, l'état après : (1) insertion de "ALLO" ; (2) insertion de "ALOHA" ; (3) insertion de "BOB" ; (4) insertion de "AL" ; (5) suppression de "ALOHA".

Chaque tuple stocké dans la structure correspond à un chemin unique dans l'arbre. Par exemple, pour le tuple "allo", il devrait y avoir un chemin qui démarre à la racine et qui passe successivement par des noeuds contenant les symboles a, l, l, o. Pour savoir si un chemin de l'arbre correspond à un tuple stocké ou non, on annote le dernier noeud du chemin en mettant est_fin = true. Si deux tuples partagent un même préfixe, alors leurs chemins commencent par les mêmes noeuds et se séparent après le préfixe. Par exemple dans la figure, "ALLO" et "ALOHA" ont le même sous-chemin pour "AL", mais ces chemins vont dans des directions différentes pour leur lettre suivante.

Pour un noeud donné v, les enfants de v sont stockés dans un map. La clé de ce map est un objet de tuple, et la valeur est le noeud correspondant à cet objet. Rappelons que les clés ne sont pas répétées, donc chaque objet mène à un noeud suivant unique (ou n'existe pas).

Vous devez implémenter les fonctions C++ suivantes :

inserer(cle) : ajoute la clé dans la structure de données, si elle n'est pas déjà présente. Le dernier noeud du chemin correspondant à la clé doit avoir est_fin à *true*. Il est possible d'insérer un tuple vide, qui compte pour une clé à stocker.

Recommandation. Ma solution a une fonction récursive qui reçoit la clé à supprimer, un itérateur *it* sur l'élément courant du tuple, et v le noeud courant. L'idée est de trouver le bon enfant où aller selon $*it$ et d'incrémenter l'itérateur.

contient(cle) : retourne *true* si la clé est présente, false sinon.

supprimer(cle) : retire la clé de la structure, si elle est présente. Les noeuds inutiles de l'arbre doivent être supprimés (un noeud est inutile s'il n'est sur aucun chemin correspondant à un élément stocké).

Recommandation. Ma solution a une fonction récursive qui reçoit la clé à supprimer, un itérateur *it* sur l'élément courant du tuple, et v le noeud courant. La fonction retourne *true* si v est devenu inutile et peut être supprimé, et l'appel parent se charge de la suppression si nécessaire.

size() : retourne le nombre de clés stockées dans la structure.

clear() : supprime tous les éléments, en nettoyant la mémoire néccessaire.

Autres fonctions : ajoutez toute autre méthode standard pour le bon fonctionnement d'une classe C++ (construction, destruction, copie, ...). Elles n'ont pas été ajoutées pour vois, à vous de déterminer ce qu'il faut. Vous pouvez aussi ajouter des méthodes privées, par exemple pour la récursion.

Partie 2 : tri de tuples

Implémentez la fonction get_tri, qui retourne un vecteur contenant toutes les clés stockées en ordre croissant. Un tuple est plus petit qu'un autre selon l'ordre lexicographique, comme dans le dictionnaire (un définition formelle se trouve plus bas).

Pour construire les objets à retourner, vous pouvez supposer que TYPESDD implémente la fonction push_back (c'est le cas pour les string, list, vector) et que TYPE implémente l'opérateur $<$.

Donnez ensuite la complexité O en temps de votre implémentation, avec une brève justification. Comme paramètres d'analyse, vous devez utiliser n le nombre d'éléments stockés, et k la longueur maximum d'un tuple. Vous pouvez donner votre réponse tout en haut du .h, ou bien dans un fichier séparé.

Définition de $<$ pour deux tuples : Étant donné deux tuples $A = (a_1, \ldots, a_p)$ et $B = (b_1, \ldots, b_q)$, on a $A < B$ si une des deux conditions suivantes est satisfaite : (1) il y a un indice i entre 1 et $\min(p, q)$ tel que $a_1 = b_1, a_2 = b_2, \ldots, a_{i-1} = b_{i-1}$ et $a_i < b_i$; ou bien (2) $p < q$ et $a_1 = b_1, a_2 = b_2, \ldots, a_p = b_p$. Par exemple, "allo" $<$ "aloha" et "bob" $<$ "bobinski".

<u>Évaluation</u>

Comme pour les autres devoirs, vous serez pénalisés pour les erreurs et imprécisions selon les critères suivants : Respect des spécifications (45 points) ; Qualité du code (45 points) ; Respect des normes de programmation (10 points).

<u>Remise du travail</u>

Vous devez seulement remettre le fichier arbrelexico.h. Ne remettez pas d'autre fichier. Il ne doit y avoir qu'une seule remise par équipe, à partir d'un seul CIP. Le nom de chaque personne doit être clairement **indiqué en entête** du fichier .h. Notez qu'il n'y a pas de script de correction automatique - votre code sera inspecté à la correction.